SISTEMA DE MONITOREO DE DISPONIBILIDAD

FastAPI | React | Gemini AI

Dashboard en tiempo real y chatbot con IA

¿Qué Construimos?

Backend con FastAPI

API REST para metricas de disponibilidad

Frontend con React + TypeScript

Dashboard interactivo en tiempo real

Chatbot con Gemini AI

Respuestas en lenguaje natural

Stack: FastAPI · React · Docker · Gemini AI

Arquitectura

1. **CSV en Google Drive**
   Fuente de datos
2. **Backend procesa y expone API**
   FastAPI
3. **Frontend consume la API**
   Graficos y dashboard
4. **Chatbot con RAG**
   Respuestas con datos reales

Transformacion Clave

PROBLEMA

Los datos CSV son puntos de cambio de estado

SOLUCION

Se convierten a intervalos de tiempo continuo

EJEMPLO

10:00 online → 10:05 offline → 10:15 online

online: 10:00 a 10:05 (5 min)
offline: 10:05 a 10:15 (10 min)

Disponibilidad = tiempo online / tiempo total

API Endpoints

GET `/api/availability/stores`

Lista de tiendas

GET `/api/availability/global`

Metricas globales

GET `/api/availability/store/{id}/intervals`

Intervalos de una tienda

POST `/api/chatbot`

Preguntas en lenguaje natural

Chatbot con RAG

1. **Extraer datos reales**
   Del CSV filtrado por tienda y fecha
2. **Enriquecer el prompt**
   Con los datos extraidos como contexto
3. **Gemini responde**
   Usando unicamente ese contexto (RAG)

Pregunta: *¿Disponibilidad de tienda 42?*
Respuesta: **94.5%**

Dashboard

Filtros
Seleccionar tienda y rango de fechas

KPIs Globales
Disponibilidad promedio, uptime, downtime

Tabla y Grafico
Vista detallada por tienda seleccionada

Chatbot integrado
Consultas en lenguaje natural

React · TypeScript · Recharts · Axios

Uso de IA

Prompt de arquitecto, no de usuario

Se definio el rol (experto en FastAPI/React), contexto (CSVs reales), estructura esperada (capas) y restricciones de calidad antes de pedir codigo.

Datos reales como contexto

Los archivos CSV fueron adjuntados al prompt para que el modelo infiriera el modelo de datos correcto.

Entrega completa, no snippets

Se pidio codigo listo para ejecutar: estructura de carpetas, todos los archivos, Dockerfile y docker-compose. Cero pseudocodigo.

GPT-5.2-Codex

Estrategico

El prompt definio arquitectura, restricciones y calidad antes de escribir una sola linea.

El output se reviso, adapto e integro

Eres un experto en ingeniería de software con especialización en Python, FastAPI, React (TypeScript) y sistemas con agentes de AI.

Te adjunto algunos archivos CSV de ejemplo que contienen eventos históricos de disponibilidad de tiendas (cambios de estado online/offline). Debes analizarlos y usarlos como referencia para diseñar correctamente el modelo de datos, el procesamiento y las métricas.

Tu tarea es generar una solución completa, profesional y lista para ejecutarse, estructurada como dos repositorios separados:

Debes generar:

1. Backend: API RESTful en FastAPI
2. Frontend: Aplicación en React con TypeScript.

 # BACKEND (FastAPI)
- Estructura de proyecto en capas (similar a arquitectura por capas en Spring Boot), incluyendo:
  - routers (controllers)
  - services (lógica de negocio)
  - repositories (acceso a datos)
  - models / schemas (Pydantic)
  - config (variables de entorno)
- Código completo de TODOS los archivos (listos para copiar y pegar)
- Procesamiento de un CSV con eventos de disponibilidad (online/offline)
- Transformaciones clave:
  - cálculo de duración de estados
  - agregaciones por tienda
  - métricas de disponibilidad
- Endpoints RESTful bien diseñados:
  - métricas
  - datos agregados
  - consulta por filtros
  - endpoint de chatbot
- Integración de un chatbot basado en LLM que:
  - responda preguntas sobre los datos
  - evite alucinaciones (basado solo en datos procesados)
- Uso de variables de entorno (.env)
- Dockerfile funcional

 # FRONTEND (React + TypeScript)
- Proyecto creado con Vite
- Estructura clara por componentes
- Código completo de TODOS los archivos
- Dashboard con:
  - gráficas (ej: Recharts)
  - tablas
  - filtros dinámicos
- Integración con el backend vía HTTP (Axios o fetch)
- Componente de chatbot conectado al endpoint del backend
- Manejo de estado limpio (hooks)
- Variables de entorno
- Dockerfile funcional

 # DEVOPS
- docker-compose.yml para levantar toda la aplicación (frontend + backend)
- Configuración clara de variables de entorno
- Instrucciones para correr el proyecto localmente

 # FORMATO DE RESPUESTA
- Muestra primero la estructura de carpetas de cada repositorio
- Luego el contenido de cada archivo (con nombre de archivo como encabezado)
- El código debe ser completo, funcional y coherente entre sí
- No omitas archivos importantes
- No uses pseudocódigo

 # REQUISITOS DE CALIDAD
- Código limpio, estructurado y en inglés con comentarios concisos.
- Buenas prácticas (arquitectura, separación de responsabilidades)
- Sin sobreingeniería innecesaria
- Explicaciones breves y solo cuando aporten valor.

Código Organizado y Buenas practicas

Backend (FastAPI)

`routers/`

Controllers — solo ruteo HTTP

`services/`

Logica de negocio desacoplada

`repositories/`

Acceso a datos (CSV/procesamiento)

`models/ + schemas/`

Pydantic para validacion y tipado

`config/`

Variables de entorno (.env)

Frontend (React + TS)

Componentes por responsabilidad

Dashboard, ChatBot, StoreTable, AvailabilityChart separados

Hooks para estado limpio

useState / useEffect — sin prop drilling innecesario

Axios con variables de entorno

VITE_API_URL configurable por entorno

✓ Docker + docker-compose

✓ Codigo en ingles + comentarios concisos

✓ Separacion de responsabilidades